Python 程式設計實務：CH2 流程控制 (Flow Control) 深度解析

授課教師：Ivan Chen 課程目標：深入掌握邏輯判斷與自動化迴圈實務，建構解決複雜問題的職場自動化思維，並學習如何撰寫具備強健性（Robustness）的工業級代碼。

2.1 程式結構與流程控制概論：程式的決策大腦

在軟體開發的世界裡，流程控制（Flow Control）被譬為程式的「靈魂」。如果說資料結構（Data Structure）是程式的軀體與血液，那麼流程控制就是決定軀體如何行動、血液如何流向的神經系統。程式不再只是死板的計算機，而是能根據環境變化做出反應的智慧工具。

這項繁複的翻譯工作（將人類意圖轉化為機器行為），主要依賴以下三種基礎結構的交織與應用：

1. **循序結構 (Sequential)**： 這是最基礎、最直觀的結構。程式碼如同建築藍圖的基礎，由上而下、不偏不倚地逐行執行。
    - **職場實務**：代表了「標準作業程序 (SOP)」的線性執行。例如：讀取報表檔 -> 驗證資料格式 -> 計算總計 -> 生成 PDF -> 寄送郵件。
    - **重要性**：確保每一道手續都按照既定順序完成。在循序結構中，順序的錯誤往往會導致邏輯災難（例如：還沒讀取檔案就嘗試計算）。
2. **選擇結構 (Selection)**： 這是賦予程式「判斷能力」的關鍵。它能依據不同的布林條件（True 或 False），讓程式在多條執行路徑中做出決策。
    - **職場實務**：實現智慧化的分流管理。例如：電商網站偵測用戶 IP，若來自海外則自動切換幣值；或系統監測伺服器負載，若超過 80% 則觸發警報並啟動備援機制。
    - **核心價值**：處理例外狀況（Edge Cases）的能力，這是專業程式碼與業餘腳本的分水嶺。
3. **重複結構 (Iteration)**： 這是發揮電腦「暴力運算」與「超高耐性」優勢的利器。它讓程式碼能以極高速度重複執行特定區塊，處理那些對人類而言枯燥、耗時且極易因疲勞而產生的勞務。
    - **職場實務**：批次化任務。例如：在一秒內比對一萬筆客戶的身分 ID，或持續監控工廠感測器數據直到數值恢復正常。
    - **效能思維**：學習如何用最少的循環次數完成任務，是工程師進階的必經之路。

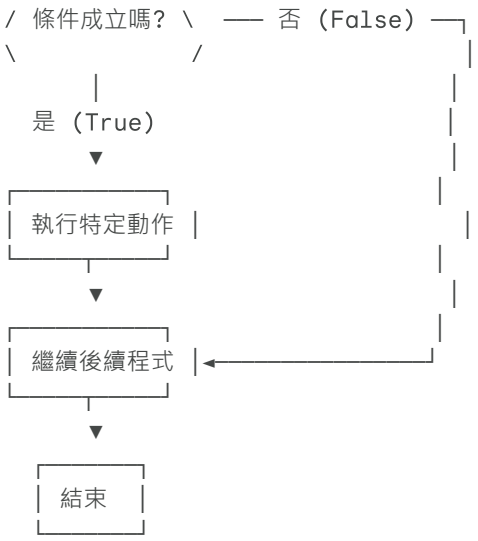
2.2 選擇結構：決策判斷的實作與深度邏輯

2.2.1 單向選擇 (Simple if)

單向選擇是最基本的攔截機制。它像是一個「品質篩檢站」：只有當物品符合特定標準時，才進行標記或處理；若不符合，則視為常規狀態，不予干預。

【邏輯流程圖：單向選擇】





【職場實務：智慧化電商折扣系統】

```
# 初始環境設定
shipping_fee = 100  # 預設基礎運費
total_amount = int(input("請輸入訂單總金額："))
membership_level = "VVIP"  # 模擬從資料庫讀取的會員等級

# 單向攔截：當條件滿足時，執行「特定優化」
# 使用 or 運算子實現多重條件觸發
if total_amount >= 2000 or membership_level == "VVIP":
    shipping_fee = 0
    print("【系統訊息】系統識別：符合滿額或尊榮會員資格。")
    print("【狀態變更】已成功套用『免運費專案』。")

# 輸出最終計算結果
print(f"\n===== 結帳明細 =====")
print(f"商品原始金額：{total_amount} 元")
print(f"應付運費支出：{shipping_fee} 元")
print(f"應付總額 (含運)：{total_amount + shipping_fee} 元")
```

🔍 **程式碼逐行詳解與技術細節：**

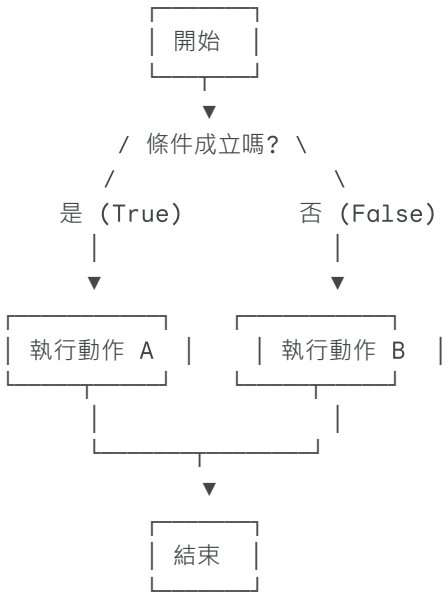
- **第 3-4 行**：我們初始化變數。注意 `int(input(...))`，這是因為 `input()` 預設抓到的是「文字」，必須轉成「整數」才能做大小比較，否則會產生 `TypeError`。
- **第 7 行**：這是核心邏輯。使用了 `or`（或）運算子，代表只要「金額大於等於 2000」**或者**「等級是 VVIP」其中一個成立，條件就會變為 `True`。這體現了商業邏輯中的「多重優惠入口」。
- **第 8-10 行 (縮排區塊)**：只有在第 7 行為 `True` 時才會執行。這裡我們將原本是 100 的運費改為 0。
- **第 13-16 行**：不管有沒有免運都會執行。這就是循序結構的特性，確保結帳資訊始終對用戶透明。

📝 **課堂即時練習 2-2-1：入職體檢警示系統 > 任務描述**：公司年度健檢系統需要自動篩選血壓異常員工。請撰寫程式讓用戶輸入「收縮壓」數值（整數）。若數值大於 140，印出「提醒：您的收縮壓偏高，請列入自主健康管理並諮詢醫師」。

2.2.2 雙向選擇 (if-else)：排他性邏輯的應用

當業務邏輯屬於「二選一」、「互補」或「排他」性質時（例如：男/女、成功/失敗、正常/異常），`if-else` 是最穩健的保險。它確保程式不會出現「邏輯真空」，任何情況都會有對應的出口。

【邏輯流程圖：雙向選擇】



【職場實務：金融業密鑰驗證與安全監控】

```
stored_key = "SECURE_PASS_2026"
user_key_input = input("請掃描身分密鑰或輸入驗證碼：")

# 使用全等運算子進行精確比對
if user_key_input == stored_key:
    print("【身分確認】密鑰比對成功。")
    print("【動作執行】正在開啟高階管理權限中心...")
else:
    # else 處理了所有不等於正確密鑰的情況（包含空白、錯字或惡意嘗試）
    print("【警告】驗證失敗！")
    print("【安全措施】已紀錄嘗試時間與來源 IP，系統將進行流量管制。")
```

🔍 **程式碼深度解析：**

- **第 5 行 (`==`)**：這是判斷相等，而非設定值的 `=`。在安全敏感的場景中，任何細微的字元差異（包含空白）都會導致 False。

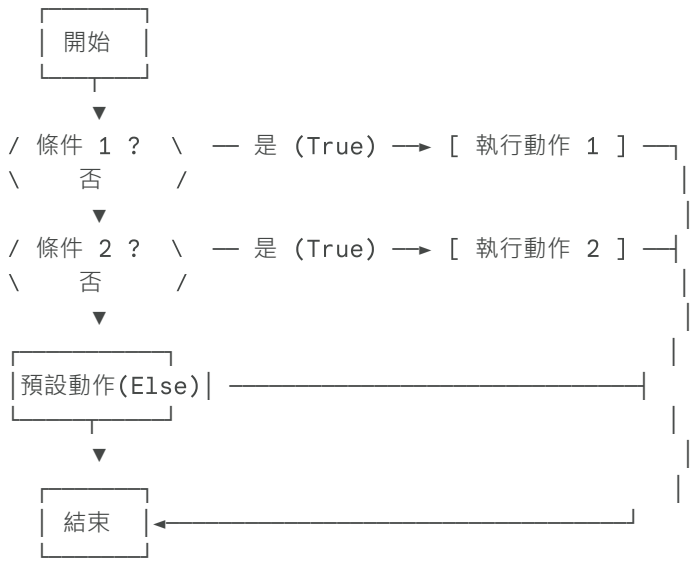
- **第 8 行 (`else:`)**：這是一個「捕捉網」。在專業開發中， `else` 區塊非常重要，因為它定義了當使用者「不按牌理出牌」時系統該如何自保。
- **安全性啟示**：永遠不要在 `if` 或 `else` 區塊中留下空白，至少要提供一個訊息或紀錄，否則系統會因無法追蹤狀態而產生安全漏洞。

> 📝 **課堂即時練習 2-2-2：面試資格初審系統 > 任務描述**：請撰寫程式讓用戶輸入「工作年資」。若年資滿 2 年 (含) 以上，顯示「符合面試資格」；否則顯示「資歷尚未達標，您的履歷已存入人才庫備查」。

2.2.3 多向選擇 (if-elif-else)：精確級距與「攔截效應」

在處理多層級利潤、績效或稅率時，我們需要精確的「過濾網」。 `elif` 允許我們在第一層過濾不通過後，進行第二次、第三次的細緻檢查。

【邏輯流程圖：多向選擇】



【職場實務：企業稅率計算與階梯利潤核發】

```
# 接收數據並進行防錯轉換 ( Defensive Programming 意識 )
try:
    revenue = float(input("請輸入部門本季營業額 (單位：萬元)："))
except ValueError:
    print("輸入格式錯誤，系統將預設為 0。")
    revenue = 0

# 建立判斷結構：注意順序必須「由大到小」
if revenue > 500:
    bonus_rate = 0.15
    tier = "頂尖 (Platinum Tier)"
elif revenue > 300:
    bonus_rate = 0.10
    tier = "優良 (Gold Tier)"
elif revenue > 100:
    bonus_rate = 0.05
    tier = "穩定 (Silver Tier)"
```

```
    else:
        bonus_rate = 0.02
        tier = "基礎 (Bronze Tier)"

    actual_bonus = int(revenue * bonus_rate * 10000) # 換算回新台幣並取整數
    print(f"\n部門評級結果：{tier}")
    print(f"依據級距 {bonus_rate*100}% 計算，本季獎金總額為：{actual_bonus} 元")
```

🔍 **技術細節與攔截效應解析：**

- **攔截順序的重要性**：多向選擇會「由上而下」執行，一旦找到第一個 True 就會跳出結構。如果我們將 `> 100` 放在最上面，那麼 `500` 的業績也會直接被 `> 100` 截斷，導致該部門領取錯誤（較低）的獎金。這在工程實務中稱為「攔截錯誤」。
- **第 13-20 行**：每一個 `elif` 都隱含了上一層的不成立。例如 `elif revenue > 300` 其實代表了 `100 < revenue <= 300`。
- **第 22 行**：使用 `int()` 是為了在最後結算時去除浮點數計算可能產生的微小尾數誤差，這是財務系統常見的處理方式。

> 📝 **課堂即時練習 2-2-3：商品折扣等級計算 > 任務描述**：請撰寫程式輸入商品單價。單價 5000 以上打 8 折；3000 以上打 9 折；其餘不打折。最後印出「折扣後單價」。

2.3 重複結構：自動化運算的基礎

2.3.1 for 迴圈與 range() 函式詳解

`for` 迴圈用於「遍歷」已知的成員。它是 Python 中最高效的迭代方式，特別適合處理清單、檔案內容或特定次數的數列。

【range() 參數解析表：精準控制序列】

語法型式	範例	產生的數列結果	說明
單參數	`range(5)`	0, 1, 2, 3, 4	從 0 開始，產生 5 個數字 (不含結束值)
雙參數	`range(1, 6)`	1, 2, 3, 4, 5	指定 [起始值, 結束值)，包含 1 不含 6
三參數	`range(2, 11, 2)`	2, 4, 6, 8, 10	指定 [起始, 結束, 步進值]，步進值可正可負

【實務案例：自動化薪資單批次加密】

```
staff_list = ["Alice", "Bob", "Charlie", "David", "Eve"]

print(">>> 啟動薪資發放自動化程序 <<<")

# 使用 enumerate 同時取得索引(序號)與內容(姓名)
for index, name in enumerate(staff_list, start=1):
    print(f"--------------------------------")
    # index:03 代表數字佔三位且補零（如 001）
    print(f"【步驟 1】讀取員工 ID: {index:03} 資料中...")
```

```
    print(f"【步驟 2】準備對 {name} 進行數據加密...")
    print(f"【步驟 3】模擬郵件已寄送至 {name.lower()}@corp.com")
```

📝 **課堂即時練習 2-3-1：整數數列產生器 > 任務描述**：利用 `for` 迴圈與 `range()` 產生從 10 到 100 之間（包含 100），所有 **10 的倍數**。

2.3.2 while 迴圈：處理未知與持續監控

`while` 迴圈的核心在於「狀態變遷」。只要目標狀態持續為真，程式就不會停止。它非常適合處理使用者互動、伺服器監控或庫存變動。

【職場實務：智慧化庫存控制系統】

```
stock_quantity = 50
print(f"系統啟動。初始庫存：{stock_quantity} 單位")

# 安全庫存水位為 10，只要庫存大於 10 就繼續運作
while stock_quantity > 10:
    print(f"\n目前庫存安全：{stock_quantity}")
    try:
        user_buy = int(input("請輸入客戶購買數量 (輸入 0 強制終止)："))
        if user_buy == 0:
            print("管理員手動終止系統。")
            break  # 強制跳出迴圈

        if user_buy <= stock_quantity:
            stock_quantity -= user_buy # 複合賦值：更新狀態，避免無限迴圈
            print(f"交易成功！庫存減少為：{stock_quantity}")
        else:
            print(f"【警示】庫存不足！現有數量：{stock_quantity}")
    except ValueError:
        print("請輸入有效的整數數字。")

print("系統提示：庫存低於安全水位，請立即進貨。")
```

🔍 **迴圈控制的關鍵邏輯：**

- **步進與變量更新**：`while` 迴圈中最常見的錯誤是忘記更新判斷條件中的變數（如 `stock_quantity -= user_buy`），這會導致程式進入「無限迴圈」。
- `break` **關鍵字**：它是程式的「緊急逃生口」，不論條件是否成立，都會立即跳出最近一層迴圈。
- **邊界檢查**：開發者必須考慮到用戶輸入大於庫存的情況，這體現了程式的強健性（Robustness）。

📝 **課堂即時練習 2-3-2：存款達標模擬器 > 任務描述**：假設初始存款為 1000 元，每月存入 500 元。請使用 `while` 迴圈計算並印出「需要幾個月」存款才會達到或超過 5000 元。

2.4 進階應用：生成式 (Comprehensions)

生成式（Comprehension）被譽為 Python 的「語法糖」，它能將傳統需要 3-4 行的「迴圈 + 判斷 + 產生新清單」縮減為一行，極大提升代碼的可維護性。

2.4.1 語法結構金律

```
新清單 = [ 運算結果 for 變數 in 來源資料 if 篩選條件 ]
```

組成部分	說明
運算結果	最終存入清單的值（如 `p * 0.9`），可在此進行函數調用或數學運算。
for 變數 in 來源	定義迭代對象，如同傳統的 for 語句。
if 篩選條件	(選用) 用於過濾資料，只有符合此條件的項目才會被處理。

2.4.2 串列與字典生成式實戰

```
# 案例 1：串列生成式 - 高薪稅務過濾
raw_salaries = [35000, 48000, 52000, 75000, 120000]
# 語法拆解：只有薪資 >= 50000 的，才進行 5% 扣稅運算
net_pay_list = [int(s * 0.95) for s in raw_salaries if s >= 50000]

# 案例 2：字典生成式 - 商品標籤映射
items = ["MacBook", "iPad", "iPhone"]
# 將商品名稱對應到其字串長度，方便後續版面佈局計算
item_layout_map = {name: len(name) for name in items}
# 結果：{'MacBook': 7, 'iPad': 4, 'iPhone': 6}
```

2.5 綜合練習：【智慧化自動倉儲結帳系統 V2.0】

練習目標：綜合運用生成式、while 互動、多向判斷與強健性設計。

【練習場景與任務】

1. **資料預處理**：建立價格清單 `[1500, 800, 3200, 450, 2100]`，利用生成式對大於 1000 元商品進行 85 折「預算處理」。
2. **互動購買**：利用 `while` 讓客戶選擇商品索引 (0-4)，輸入 `-1` 結帳，並包含 `try-except` 防止輸入錯誤字串。
3. **結算優惠**：若總額 >= 5000，結帳再折 500 元；若 >= 3000，結帳再折 200 元。

```
# 1. 預處理：會員特惠價計算
original_prices = [1500, 800, 3200, 450, 2100]
member_prices = [int(p * 0.85) if p > 1000 else p for p in original_prices]

print(f">>> 歡迎光臨！今日特價清單：{member_prices}")
total_bill = 0

# 2. 互動選購迴圈
while True:
```

```
        try:
            choice = int(input("\n請輸入購買商品索引 (0-4)，結帳按 -1："))
            if choice == -1:
                break
            if 0 <= choice <= 4:
                selected_price = member_prices[choice]
                total_bill += selected_price
                print(f"【成功加入】目前總累計：{total_bill} 元")
            else:
                print("【警告】無此編號，請重新輸入 0~4 之間的數字。")
        except ValueError:
            print("【輸入錯誤】請務必輸入整數編號！")

    # 3. 結帳折扣階梯
    if total_bill >= 5000:
        total_bill -= 500
        msg = "套用滿 5000 折 500 週年慶優惠"
    elif total_bill >= 3000:
        total_bill -= 200
        msg = "套用滿 3000 折 200 滿額禮"
    else:
        msg = "未達額外加碼門檻"

    print(f"\n===== 最終結帳明細 =====")
    print(f"優惠狀態：{msg}")
    print(f"【實付總金額】：{total_bill} 元")
```

職訓師重點筆記（Ivan's Key Takeaways）：

1. **縮排即邏輯 (Indentation is logic)**：冒號 `：` 之後的縮排決定了程式碼的從屬關係。在職場中，縮排錯誤（IndentationError）是最不該犯的低級錯誤。
2. **型別轉換的嚴謹性**：`input()` 取得的是字串，計算前必轉 `int` 或 `float`。處理數值前先想好：這筆資料需不需要小數點？